Financial Contract Combinators in the Cost-Accounted Rho Calculus

An executable, resource-safe settlement language — and a firmer footing for DeFi

L. G. Meredith
F1R3FLY.io — lgreg.meredith@f1r3fly.io

June 2026

Abstract

Peyton Jones, Eber and Seward showed that a small set of combinators can describe an unbounded family of financial contracts, and that a compositional denotational semantics can value them. Their language describes *what a contract is* and *what it is worth*; it does not say how a contract *executes* on a shared, adversarial, resource-metered substrate, nor what guarantees that execution enjoys. We close that gap by realising the combinator language in the cost-accounted rho calculus: the reflective higher-order process calculus that is the formal core of Rholang, equipped with the cost endofunctor that folds resource accounting into the grammar. Reflection supplies the substrate — contracts, observables and obligations are all processes, and names are quoted processes — while cost accounting supplies three things a settlement language needs and a denotational semantics cannot: *located authority* over funds, *atomicity* of multi-step settlement, and a *linear resource proof* that a deployment is funded *before* any step fires. We re-express five instrument families in this calculus — the core combinators, loans, a Dutch auction, cash-settled futures, and a book of fourteen power-market instruments — and in doing so identify a correction to the original cost presentation (token stacks must be *located*, because parallel composition is associative–commutative) and a structural finding (the combinator `and` silently conflates a simultaneous atomic swap with a bundle of independently-scheduled obligations; the acceptance gate forces the two apart). We argue that the resulting discipline addresses, by construction, several of the failure modes that have dogged decentralised finance — stranded funds, ambient authority, gas exhaustion mid-transaction, and post-hoc conflict resolution — that it lightens the post-2008 regulatory burden by making the audit trail native and compliance a proof discharged before execution rather than an audit performed after it, and that these same properties make it a credible internal settlement substrate for incumbent finance. We close by looking forward to the spatial-behavioral types that the calculus's own logic generator, OSLF, makes available as a machine-checkable discipline for financial computing.

Contents

1 Introduction

A financial contract is an agreement to exchange cash flows contingent on time and on observable quantities. Peyton Jones, Eber and Seward [1] made the seminal observation that the sprawling catalogue of traded instruments — swaps, futures, caps, floors, options of every flavour — is generated by a handful of combinators, and that giving those combinators a compositional denotational semantics lets one *value* an arbitrary contract by structural recursion. Their combinators (`zero`, `one`, `give`, `and`, `or`, `scale`, `truncate`, `get`, `anytime`) and the derived forms built from them (`zcb`, `european`, `american`) remain the canonical answer to the question "what is the right vocabulary for financial contracts?"

That work answers two questions — what a contract *is*, and what it is *worth* — and deliberately leaves a third untouched: how a contract *executes*. On a single trusted machine, execution is unremarkable. On a distributed ledger — where execution is concurrent, adversarial, and metered, where a half-completed settlement can strand value, where the right to move funds is a capability that must not leak, and where running out of fuel mid-transaction is a real and frequent event — execution is the entire problem. This is the regime of decentralised finance (DeFi), and it is precisely the regime in which the combinator language has the most to offer and has been least applied.

We supply the missing layer by realising the combinator language in the *cost-accounted* rho calculus [5], a calculus obtained from the reflective higher-order rho calculus [3] — the formal core of Rholang [7] — by applying the cost endofunctor of [6]. The thesis of this note is simple to state:

> *The combinator language, interpreted in the cost-accounted rho calculus, is not merely a description language with a valuation semantics; it is an executable settlement language whose executions are resource-safe and atomic by construction.*

We make the case constructively, by re-expressing five instrument families and reading off what the apparatus buys in each.[1] Two findings emerge along the way that are of independent interest. First, a correction: the original rho calculus presentation of cost accounting left token stacks floating in the parallel-composition bag, and because that composition is associative–commutative this is an *ambient-authority* leak; the fix is to *locate* every stack on an unforgeable channel — in this calculus a location is just a name — so that authority is possession of the name, not adjacency in the bag (§3.2). Second, a structural observation: the combinator `and` is overloaded, denoting both a simultaneous bilateral swap and a bundle of independently-scheduled obligations, and the cost calculus's acceptance gate forces these apart because the gate's scope *is* the unit of financial atomicity (§4.2).

Outline. §2 recalls the rho calculus, the combinators, and the cost construction. §3 is the realisation: the contract protocol, located funding, and the key features that distinguish a settlement language from a description language. §4 surveys the five families and the four cost-accounting patterns they exhibit. §5 argues the case for DeFi. §8 looks ahead to spatial-behavioral types. §9 concludes.

2 Background

2.1 The reflective higher-order rho calculus

The rho calculus [3] is a process calculus in which *names are quoted processes*. Its grammar is

$$P, Q \;::=\; 0 \;\mid\; \texttt{for}(y \leftarrow x)\, P \;\mid\; x!(Q) \;\mid\; P \mid Q \;\mid\; *x, \qquad x, y \;::=\; @P,$$

with input $\texttt{for}(y \leftarrow x)P$, output $x!(Q)$, parallel composition $P \mid Q$, the quotation $@P$ of a process to a name, and the dereference $*x$ recovering the process a name quotes. Communication substitutes the quoted datum for the bound name. Reflection — the $@\,/\,*$ pair — is what makes the calculus higher-order without a separate function space, and it is what lets contracts, observables, oracles and obligations all be *processes*, exchanged as *names* on channels and run by dereference. This is the substrate on which Rholang and the F1R3FLY platform are built.

2.2 The combinators

A contract, in the realisation below, is a channel `C` that, when sent an acquisition message, discharges the rights and obligations it denotes. Following [1] the primitives are `zero` (no rights or obligations), $\texttt{one}(k)$ (acquire one unit of currency k), $\texttt{give}(c)$ (reverse the parties of c), $\texttt{and}(c_1, c_2)$ (acquire both), $\texttt{or}(c_1, c_2)$ (the holder chooses one), $\texttt{scale}(o, c)$ (scale c by an observable o), $\texttt{truncate}(t, c)$ (let c expire at t), and $\texttt{get}(c)$ (acquire c at its horizon). An *observable* is a time-varying quantity both parties agree on, modelled as a persistent channel that returns its current value. From these one derives the zero-coupon bond $\texttt{zcb}(t, x, k) = \texttt{scale}(x, \texttt{get}(\texttt{truncate}(t, \texttt{one}(k))))$, the European option $\texttt{european}(t, u) = \texttt{get}(\texttt{truncate}(t, u\ \texttt{or}\ \texttt{zero}))$, and so on, exactly as in the source.

2.3 Cost accounting as a construction on the calculus

Rholang gates every deployment behind a *phlogiston* balance tied to a signature. The cost papers [5, 6] show that this gating is not an ad hoc runtime extension but the image of a generic endofunctor

[1] The full implementation — rho source for all five instrument families re-expressed in the cost-accounted calculus — is open-sourced in the F1R3Fi repository [23].

C on a category of interactive process theories. The construction adjoins, to the host calculus, three sorts and a discipline:

- *Signatures as names.* The name sort is extended, $x, y ::= @T \mid s$, so a signature s is itself a channel. A signature both *funds* the phlogiston of a communication and *authorises* the underlying value movement; there is no separate translation layer [5, §5.1]. Signatures compose: $s_1 * s_2$ is joint (multi-signature) authority.
- *Token stacks as first-class terms.* A stack $S ::= () \mid s :: S$ is a temporal monoid; its head is the next token to spend. A balance is a stack; minting and delegation are sends of stacks on signature channels.
- *Wrapping by construction.* Every continuation is re-sorted to a wrapped-term sort: a signed thunk $\{P\}_s$ that fires only by spending a matching token. "No redex fires without being unwrapped" is then a sorting invariant preserved by reduction [6, §3], not a property to be re-established by traversal; metering is lazy, charging for work performed rather than exposed.

The single communication rule of the host is replaced by a gated family, of which the base case is

$$\{\texttt{for}(y \leftarrow x)\, T \mid x!(U)\}_s \;\mid\; s :: S \;\rightsquigarrow\; T\{@U/y\} \;\mid\; S,$$

"a communication fires only when a matching token is present, and the consumed token is released as residual." [5, §4.6] Two consequences organise everything below. First, the `for`-comprehension *is* the transaction primitive: its binding specification is the transaction, the substitution is the witness, and application to the continuation is the state update; a single communication is atomic by construction [5, §8.1]. Second, *financial* atomicity — that a multi-step operation completes entirely or not at all — is *not* a theorem of the rewrite rules but is secured by a static analysis at deployment-acceptance time: a *linear resource proof* verifying $\Sigma_s \geq \Delta_s$, supply at least demand, for every signature, *before* any step executes [5, §8.5].

3 Realising the Combinators

3.1 The cost-accounted contract protocol

A contract becomes a channel acquired by

$$\texttt{C!}(\, s_H,\ s_{Cp},\ hAddr,\ cAddr,\ fund,\ res\,),$$

where s_H and s_{Cp} are the holder and counterparty signatures (names that are simultaneously capabilities and authorisations), $hAddr$, $cAddr$ are the parties' ledger addresses, *fund* is a *located funding slot* (below), and *res* returns $(\texttt{true}, \texttt{Nil})$ or $(\texttt{false}, err)$. The combinator `give` swaps (s_H, s_{Cp}) and $(hAddr, cAddr)$ together: swapping *which signature funds* each leg is exactly the two-sided surface of the interaction cut [5, §4.8.2] made operational. The payment primitive `one` is funded by the payer, drawing one token; the structural combinators forward the funding slot to their sub-contracts. Thus *cost incidence follows economic incidence*: the party who pays the cash flow pays the phlogiston that moves it — a property that is simply inexpressible in a model with a single fee-paying deployer.

3.2 Located token stacks: a correction

The first realisation attempt exposes a defect in the original cost presentation. There, a token stack $s :: S$ is written into the ambient process soup and consumed by whatever communication needs

it. But parallel composition is associative–commutative: the soup is an unordered bag, and *any* process in it is adjacent to *every* stack in it. Taking adjacency as the right to spend is *ambient authority* [15] — the antithesis of an object-capability discipline, and a latent double-spend.

The fix is to *locate* every stack, and in the rho calculus a location is nothing more exotic than a *name*. To locate a token stack is simply to hold it on a channel: the stack becomes a message $c!(\, s :: S \,)$ on a channel c, and the right to spend from it is exactly the right to receive on c. Because channels minted by `new` are unforgeable, possession of c is a genuine object capability — a process that was never handed c cannot name it, cannot receive on it, and so cannot touch the stack, even though, the bag being associative–commutative, it sits in the same soup. The name does the work that position cannot: it says *who may draw* (possession of c), while the signature at the head of the stack says *which token is spent* (the guard). Authority is possession of a name, never adjacency. In the realisation every fuel stack lives on such a channel; none floats free.

This is the concrete, rho-calculus reading of the general *located resource stack* $S(I, -)$ of [6, §10.4], and the abstraction collapses pleasantly here. In general the location I may be carried either structurally, by rigid position, or nominally, by an explicit name; but an associative–commutative combinator like | destroys position, so in this setting the location can *only* be a name. What the general construction calls an "interaction surface" is, for us, just a channel. The same mechanism is the *funding slot* of [5, §5.7]: an open contract — an auction whose bidders are unknown in advance — creates a fresh channel and accepts fuel on it, funded by whoever holds the channel, with no need to name them at compile time.

3.3 The acceptance gate and financial atomicity

The motivating hazard of the cost papers is a payment decomposed into a debit and a credit that funds the debit but not the credit, stranding value in transit [5, §8.2]. Every multi-step settlement in the combinator language has this shape, and the apparatus closes it the same way: a multi-step acquisition first *reserves* its entire demand from a located slot, all-or-nothing, before any leg fires. Listing 1 shows the reservation gate; it is the operational content of the linear resource proof $\Sigma_s \geq \Delta_s$, decided before execution. If the reservation fails the slot is left untouched and nothing moves; if it succeeds, every step draws from the committed purse and cannot stall for lack of fuel.

Listing 1: The acceptance gate: an atomic reservation from a located slot, the operational form of the linear resource proof.

```
// reserve(L, sig, n, ret): remove the first n tokens of `sig`
// from located slot L, all-or-nothing; either (true, purse)
// with a committed private purse, or L is left UNTOUCHED.
contract reserve(@L, @sig, @n, ret) = {
  for (@stack <- @L) {
    new takeCh, peel in {
      contract peel(@rem, @k, @acc, pret) = {
        match k {
          0 => pret!((true, acc, rem))
          _ => match rem {
            [] => pret!((false, Nil, Nil))
            [hd ...tl] => match hd == sig {
              true  => peel!(tl, k - 1, acc ++ [hd], *pret)
              false => pret!((false, Nil, Nil)) } } } } |
      peel!(stack, n, [], *takeCh) |
      for (@result <- takeCh) {
        match result {
          (true, taken, remainder) => {
```

```
            @L!(remainder) | new P in { P!(taken) | ret!((true, *P)) } }
          (false, _, _) => { @L!(stack) | ret!((false, "insufficient fuel")) }
        } } } }
}
```

Remark 3.1 (Two atomicities)**.** Database-transaction atomicity — a single communication either fires or does not — is a theorem of the rewrite rules. Financial-transaction atomicity — a multi-step settlement completes or does not begin — is secured by the gate above, at the deployment boundary. The combinator tree is the unit over which demand is summed; acceptance is the unit of financial atomicity. This is the precise sense in which the description language becomes a settlement language.

3.4 The combinator `and` as an atomic join

The combinator `and` acquires both sub-contracts. Naively encoded, its two legs are funded independently, so one leg's transfer may commit while the other stalls — a half-settled swap with no rollback. The cost calculus offers the principled remedy: fund the join under the compound authority $s_1 * s_2$ and reserve the joint demand atomically, so that, by the join cost schema [5, §4.8], the redex either gathers a full token's worth of every participant's authority or does not fire. "The partial-funding hazard . . . is structurally impossible within a join." For a same-funder bundle (a fixed-payment strip) the joint reservation is from a single located purse; for a cross-party swap (a loan sale, or the bilateral settlement below) it is a two-slot reservation that funds both parties or neither.

3.5 Data-dependent demand: conservative bound and refund

The choice combinators (`or`, `european`) and the temporal combinators (`get`, `truncate`) branch on data — the holder's choice, or the block height — so which interactions fire, and hence how many tokens of which signature are drawn, depends on the run. For these the demand is not a fixed sum. The discipline is conservative: reserve the worst case over branches and refund the unforced remainder after the run [5, §8.6]. Over-funding is a benign inefficiency reconciled by a refund; under-funding would reintroduce the stranding hazard, so we never under-fund. The realisation tracks each combinator's demand as a conservative bound (max over branches for choice; the live branch for time) and the deployment boundary refunds what the run did not force.

3.6 Cost is not amount

A subtlety that the prism metaphor of [5, §5.6] makes precise: phlogiston counts *funded rendezvous*, not the magnitude of value moved. Scaling a contract by a large observable multiplies the currency transferred but leaves the fuel demand at one — one transfer, one token, whatever its size. The static funding proof concerns the count of rendezvous, not the REV moved; conflating them (as a gas-per-byte intuition invites) is a category error the apparatus forbids.

4 Five Families, Four Patterns

We re-expressed five instrument families [23]: the core combinators; loans (short, coupon and amortising, with an atomic loan sale); a Dutch auction; cash-settled futures; and a book of fourteen power-market instruments (PPA, futures, swing, interruptible supply, capacity, REC forward,

Pattern	Cost-accounting treatment	Instruments
(A) Bilateral cash settlement	Net the two opposing legs into one transfer; reserve one token from each party, refund the unused side	Futures, virtual PPA, spark spread, CRR, weather derivative
(B) Single-funder fixed leg	One funded transfer, demand 1; `give` swaps the funder	`one`, `zcb`, REC forward, capacity, PPA period
(C) Conservative-bound option	`or`/`european`: demand = max over branches; writer pre-funds the exercise case, refunded on lapse	Swing, frequency regulation, interruptible, option legs of DR / spinning reserve
(D) Time-separated bundle	Legs at different blocks and/or funded by different parties; each leg its own funded deployment under concurrent acceptance	Strips (PPA, swing, freq, forward curve); premium+option pairs (DR, spinning reserve); loan disburse+repay

Table 1: The four patterns that cover all the realised instruments.

virtual PPA, demand response, frequency regulation, spinning reserve, forward-curve strip, spark spread, weather derivative, congestion revenue right). Across all five families the instruments fall into four cost-accounting patterns, summarised in Table 1.

4.1 Bilateral settlement and the netting incentive

The futures contract is the sharpest case. Its denotation is a bilateral obligation — the counterparty pays the holder the spot price, the holder pays the counterparty the agreed price — written $\texttt{and}(\texttt{scale}(\mathit{spot}, \texttt{one}(1)), \texttt{give}(\texttt{one}(F)))$. As two gross transfers in opposite directions, funded by different parties, this is the partial-funding hazard at its worst. Cost accounting makes the gross form strictly worse on both axes it tracks: two transfers can half-settle at the ledger layer, and they cost two tokens. The cost-accounted form therefore *nets*: sample the difference at expiry and perform a *single* transfer in the appropriate direction (Listing 2). A single transfer is atomic by construction and costs one token regardless of magnitude. The settlement direction is data-dependent, so we reserve one token from each party and refund the unused side. Netting is thus not merely an optimisation but the form the apparatus rewards: it collapses a bilateral swap to a single atomic step and removes the half-settlement hazard entirely. Five of the fourteen power instruments — the genuine contracts for differences — settle through this one mechanism, parameterised by a (free) net observable.

Listing 2: Netted bilateral settlement: one transfer in the data-dependent direction; atomic by construction.

```
// at expiry T, net = (what holder receives) - (what holder pays)
match net == 0 {
  true  => res!((true, Nil))                 // flat: nothing moves
  false => match net > 0 {
    true  => { spend!(cpPurse, *sp) | ...     // cp pays holder net
               transfer!(RevVault, cAddr, hAddr, net, sC, *res) }
    false => { spend!(holderPurse, *sp) | ... // holder pays cp -net
               transfer!(RevVault, hAddr, cAddr, 0 - net, sH, *res) }
  } }
```

DeFi failure mode	Cost-accounted remedy
Stranded funds / half-settled multi-step transactions	The acceptance gate funds the whole transaction or rejects it before any step fires; netting collapses bilateral swaps to a single atomic transfer (§3).
Ambient authority / unauthorised draws on pooled funds	Located resource stacks: fuel and value sit on unforgeable surfaces; authority is possession of the surface, never adjacency in the AC bag (§3.2).
Gas exhaustion mid-execution	A deployment is accepted only if a linear resource proof certifies it is fully funded; an accepted deployment cannot run out of fuel. Overestimates are refunded.
Reentrancy	The unit of state update is a single `for`-comprehension, atomic by construction; there is no partial intermediate state to re-enter, and capabilities are not ambient.
Post-hoc conflict resolution / MEV from ordering	Concurrent acceptance over disjoint token pools: deployments signed by different signatures cannot conflict and compose without merge analysis; competing deployments are competing linear-proof obligations, at most one of which succeeds [5, 20, §8.8].
Opaque, unauditable contract logic	Contracts are declarative compositions of a fixed combinator set with a denotational valuation [1] and an operational settlement semantics; the same term is valued, executed, and (§8) typed.

Table 2: DeFi failure modes and their structural remedies in the cost-accounted realisation.

4.2 The overloading of `and`

The power book reveals that `and` silently denotes two different things. In the futures it is a *simultaneous* swap: both legs settle at the same instant and must be jointly atomic. In a demand-response contract it bundles a premium paid *now* (grid to data centre) with an option settled *later* (data centre to grid, on exercise) — two obligations at different times, funded by different parties. The denotational semantics is indifferent to the distinction; the cost calculus cannot be, because the acceptance gate's scope *is* the unit of financial atomicity. Fusing a time-separated bundle into a single atomic gate would over-reserve fuel and wrongly couple two obligations that should stand or fall independently. The realisation therefore splits `and` into an atomic join (Pattern A/C settlements) and a bundle returned as a list of independently acquirable legs (Pattern D), each its own funded deployment under *concurrent acceptance* [5, §2.3]. That this distinction is invisible to valuation but forced by execution is, we think, a small vindication of the thesis: the settlement semantics sees structure the denotational semantics cannot.

5 A Firmer Footing for DeFi

Decentralised finance has repeatedly stumbled over a small set of failure modes that are not failures of cryptography or consensus but of *execution discipline* [18]. The cost-accounted combinator language addresses each of them not by patching but by construction. Table 2 summarises the correspondence.

The deeper point is methodological. Smart-contract platforms typically offer a Turing-complete imperative language and a gas meter bolted on outside it, leaving atomicity, authority and resource sufficiency as the programmer's responsibility — which is to say, as the auditor's nightmare. The cost-accounted rho calculus inverts this: resource accounting is *inside* the grammar ("valu-

able friction at the syntactic level" [5, §5.3]), authority is an object-capability discipline carried by unforgeable names, and atomicity is either a theorem (single step) or a statically checked proof obligation (multi-step). Building a financial DSL — the combinator language — on *this* substrate means the instruments inherit those guarantees rather than re-litigating them. DeFi does not need a more clever contract; it needs a substrate on which the obvious contract is already safe.

6 Regulatory Compliance and Internal Deployment

The 2008 financial crisis was, in large part, a crisis of opacity: derivative exposures that no party could see whole, contracts no system could value in aggregate, and obligations whose interlocking was discovered only as they failed. The regulatory response built a standing apparatus of reporting, recordkeeping, clearing, and capital discipline whose cost the industry now carries continuously. In the United States the Dodd–Frank Act [24] (Title VII) mandated central clearing of standardised swaps, real-time public reporting, and the reporting of every swap to a registered swap data repository; in the European Union, EMIR [25] imposed parallel trade-reporting and clearing obligations together with margin requirements for uncleared derivatives, and MiFID II / MiFIR [26] added exhaustive transaction-reporting and recordkeeping duties; Basel III [27] tightened risk-based capital, leverage, and liquidity ratios; and the BCBS–IOSCO uncleared-margin rules made collateralisation of non-cleared positions mandatory. The domain these regimes police — swaps, futures, options, structured payments — is exactly the domain of the combinator language. The cost-accounted substrate supports their obligations along two axes: a *record* that is native rather than reconstructed, and a *proof* discharged before a contract executes.

The record axis. A distributed ledger is, by construction, a complete, timestamped, tamper-evident history of every deployment and settlement. Here that record is unusually faithful, because a contract is not opaque bytecode but a declarative combinator term with a denotational valuation and an operational settlement trace: the record *is* the contract, together with the exact sequence of funded rendezvous that settled it. The audit trail that swap-data reporting [24], EMIR trade reporting [25], and MiFID II transaction reporting [26] demand at significant standing cost is, on this substrate, a by-product of execution rather than a separate reporting pipeline bolted alongside it. Counterparty identity — the contribution of the post-crisis Legal Entity Identifier regime to transparency — is carried directly by the signatures-as-names that already fund and authorise each leg.

The proof axis. The deeper opportunity is to discharge compliance *statically*, before a contract is accepted, rather than auditing it afterwards. Because contracts are declarative terms with formal semantics, and because the calculus already checks one resource property — the linear funding proof of §3 — at the deployment gate, the same gate can carry regulatory proof obligations. Several map cleanly:

- *Collateralisation and margin.* The uncleared-margin requirement that a position be backed by sufficient collateral is, almost verbatim, the located-purse resource-sufficiency proof of §3: a contract that cannot prove it is funded does not execute. Margin compliance is the funding proof under another name.
- *Authorised-party gates (KYC/AML).* A compound signature $\{\, \mathit{transfer} \,\}_{s_{\text{party}} * s_{\text{compliance}}}$ makes a compliance officer's co-signature a *structural* precondition of settlement [5, §2.4]: the transfer

cannot fire without it. Authorisation is a capability the term carries, not a check performed after the fact.

- *Position and exposure limits.* The spatial types of §8 bound how obligations may compose and how much of a resource a sub-contract may claim — the type-level form of a position limit or a concentration cap.

This shifts compliance from after-the-fact audit — the expensive, error-prone posture the post-2008 regime largely institutionalised — to a proof checked before any value moves: the regulatory analogue of the resource-sufficiency guarantee. As with the type discipline generally (§8), the record axis is available today, while the full proof axis builds on the type layer still in progress; the substrate is designed to carry both.

Internal deployment for incumbent finance. None of these properties — atomic settlement, capability-confined authority, a native audit trail, pre-trade compliance proofs, concurrency without post-hoc reconciliation — depends on a public, permissionless deployment. They are equally available, and arguably land first, inside a regulated institution running a private or permissioned shard as an internal settlement-and-compliance engine: the same combinator contracts, the same guarantees, without the regulatory uncertainty that still surrounds public decentralised finance. For a bank or a fintech the value proposition is concrete and quantifiable — the audit trail and pre-trade compliance proofs attack a known, measured cost line — which is often where new infrastructure earns its first production deployment. The arrival of dedicated crypto-asset regimes such as the EU's MiCA [28] makes a substrate with compliance built in, rather than bolted on, especially timely. The cost-accounted rho calculus is thus not only a firmer footing for open DeFi but a credible internal infrastructure for the very institutions that the post-2008 reforms now bind.

7 Related Work: Marlowe and the Substrate Question

The closest prior art is Marlowe [21], the financial-contract DSL developed for Cardano in the same Peyton Jones–Eber lineage as the present work [1]. Marlowe shares our premise — that financial contracts deserve a small, analysable, special-purpose language rather than a general imperative one — and pursued analysability aggressively: the language is deliberately finite and non-Turing-complete, so that the behaviours of a contract can be enumerated and checked exhaustively. It is instructive to ask why an approach so close in spirit did not become the substrate for open decentralised finance, because the answer locates precisely where our bet differs.

The difficulty was not in the contract language but in the substrate beneath it. Cardano's Extended UTXO model [22] places application state in unspent outputs, and an output may be consumed by at most one transaction per block. Any *contended, shared* state — an order book, a liquidity pool, an auction, any genuinely multi-party venue — therefore becomes a serialisation point: concurrent participants produce conflicting transactions, all but one fail, and applications recover throughput only by interposing off-chain "batchers" that order and submit on users' behalf. But most of open DeFi *is* contended shared state, and batching reintroduces exactly the centralisation and ordering discretion — a privileged sequencer, a private mempool — that a public ledger was meant to dissolve [20]. The substrate fought the workload.

The rho calculus begins from the opposite commitment. It is a process calculus; parallel composition is primitive, not an optimisation; RSpace is a concurrent tuplespace; and the cost construction's governing motivation is to replace run-to-completion with *concurrent acceptance* over disjoint token pools [5, §2]: deployments signed by different signatures provably cannot conflict and compose

without post-hoc merge analysis, while deployments competing for the same funds are competing linear-proof obligations of which at most one succeeds [5, §8.8]. Where EUTXO makes shared-state concurrency the hard case to be worked around, the cost-accounted rho calculus makes it the default case, decided structurally at acceptance time. This is the single strongest reason to expect a different outcome, and it is independent of the contract language sitting on top.

The two designs also recover safety by opposite routes, and this is the second difference. Marlowe buys exhaustive analysability by sacrificing expressiveness: a finite, non-Turing-complete language cannot host the open-ended, composable, user-defined instruments that drove adoption on more expressive platforms. We keep a universal substrate — the rho calculus is Turing-complete — and recover safety *structurally* rather than by restriction: atomicity by construction, located object-capability authority, the linear resource proof checked before execution, and the spatial-behavioral type discipline of §8. The aim is Marlowe-grade guarantees without Marlowe's expressiveness ceiling, with resource accounting inside the grammar rather than bolted on as an external fee model. We are candid that this is in part a promissory note: the operational guarantees are in hand in the realisation above, but the type-system layer that would make them statically checkable is the work of §8, still ahead of us. A process calculus also pays for its concurrency with a larger reasoning burden than a finite language — interleaving and nondeterminism are real — which is exactly why we regard the behavioural type discipline as load-bearing rather than ornamental. Other efforts to give contracts a process-calculus footing, such as BitML for Bitcoin [19], share the conviction that contracts deserve a calculus; they differ in target and do not address the concurrent-settlement problem that is our focus.

Finally, candour about traction itself. Whether a contract language is adopted is not a purely technical verdict; network effects, liquidity depth, and timing dominate, and Cardano's broader DeFi ecosystem was thin and late relative to where capital had already pooled. We do not claim a technical argument settles an economic outcome. The defensible claim is narrower and, we think, sound: the cost-accounted rho calculus removes the specific *architectural* reason a Peyton Jones–Eber contract language could not host real decentralised finance on its predecessor substrate — it makes the obvious financial contract safe, concurrent, and composable at once, where EUTXO could deliver at most two of the three.

8 Looking Forward: Spatial-Behavioral Types

The guarantees above are secured operationally — by located slots, the acceptance gate, and the conservative-bound discipline. The natural next step is to make them *types*: machine-checkable assertions a contract carries and a checker verifies. The calculus already generates the logic for this. OSLF [4] auto-generates, from a graph-structured theory, a Hennessy–Milner-style logic [10] that is adequate for bisimulation; run on the *cost-accounted* theory it yields a *graded* logic whose modalities are indexed by the signature consumed [6, §7,§11]. Two connective families suffice to express a token discipline as a type:

- *Spatial* types $K(\varphi_1, \varphi_2)$ read parallel composition as a separating conjunction in the sense of separation and spatial logics [13, 12, 14]: a term satisfies $K(\varphi_1, \varphi_2)$ when it splits into disjoint parts satisfying φ_1 and φ_2. This is the bookkeeping of ownership and duplication — whether two obligations hold disjoint funds, whether a token is copied.
- *Modal* types $\langle K(\dots)\rangle\varphi$ are the graded modalities of consumption: a term can take a funded interaction, after which φ holds.

Together they express the lattice of resource disciplines — linear (spend exactly once), affine, relevant, copyable — as *checkable* assertions, decidable when stacks are finite and located.

To see the two families at work — and that the K they instantiate need not be the same one — take the netted future of §4 — the bilateral contract for differences whose `and` is a simultaneous swap (§4.2). Its fuel sits on two located surfaces (§3.2): a holder purse drawable by s_H and a counterparty purse drawable by s_{Cp}. Instantiate the *spatial* K as parallel composition, $|$, and write purse_s for "a located stack keyed to s sits here." The ownership type is a separation,

$$\Phi_{\text{sep}} \;\equiv\; \mathsf{purse}_{s_H} \mid \mathsf{purse}_{s_{Cp}},$$

satisfied when the term splits into two disjoint purses sharing no sub-term: the two parties' funds provably do not alias — the static no-double-spend. Instantiate the *behavioural* K, by contrast, as the all-or-nothing *join* cut of [5, §4.8] — a *different* constructor, the settlement rendezvous rather than the spatial bag — and grade its modality by the signature it draws. The atomicity type is a single compound-graded diamond,

$$\Phi_{\text{settle}} \;\equiv\; \langle\,\text{join}\,\rangle_{s_H * s_{Cp}}\,\mathsf{Flat},$$

read: the contract takes one funded step whose demand is the compound token $s_H * s_{Cp}$ — one cell from each party, drawn jointly — after which the position is Flat (settled, nothing stranded). Because that compound cell may not be partially consumed [5, §4.8.5], the diamond cannot half-fire. The two predicates read the two faces of the one GSLT constructor [6, §4]: the spatial $K = |$ records *what is near what* (the purses are apart), the behavioural $K = \text{join}$ records *what is spent* (one compound step settles them). The point is sharpened by the contract that fails to type: the *gross*, un-netted future admits no single such diamond, only the staged chain $\langle\text{cut}\rangle_{s_{Cp}}\langle\text{cut}\rangle_{s_H}\,\mathsf{Flat}$, whose reachable mid-state between the two diamonds is exactly the half-settled, stranded position. Netting is what makes Φ_{settle} available, and the half-settling contract does not type-check against it.

For financial computing the payoff is concrete and, we believe, considerable. The future above is the contract-for-differences case — Φ_{settle} types its bilateral atomicity, Φ_{sep} its no-double-spend by separation — and the same two families reach further. An option writer's obligation can be typed so that the worst-case exercise is provably funded for the life of the option. A loan's repayment schedule can carry a per-instalment resource-sufficiency proof, factored by located purse into independent conjuncts [6, §12.2], so that funding a hundred-period amortisation is a hundred small local checks rather than one global one. In each case the property that the present realisation secures operationally becomes a type the term carries and a checker discharges — the financial analogue of memory safety. We regard this as the most promising direction the present work opens: not safer contracts written more carefully, but a type discipline under which the unsafe contract does not type-check.

This sits within a longer arc. The same spatial/temporal structure that underwrites the cost calculus — the spatial monoid K of what is near what, the temporal monoid :: of what is spent next — is the structure on which the calculus's authors have noted suggestive correspondences to physical field theories [6, §13]. We do not lean on those here; we note only that a financial settlement layer, a behavioural type system, and a process-theoretic account of resource flow are, in this setting, three readings of one object.

9 Conclusion

We have realised the Peyton Jones–Eber–Seward combinator language in the cost-accounted rho calculus, and in doing so turned a description-and-valuation language into an executable settlement

language whose executions are resource-safe and atomic by construction. Reflection supplies the substrate on which contracts, observables and obligations are uniformly processes; the cost endofunctor supplies located authority over funds, atomicity of multi-step settlement, and a linear resource proof checked before execution. Re-expressing five instrument families — combinators, loans, a Dutch auction, futures, and a fourteen-instrument power book — surfaced a correction (token stacks must be located, because parallel composition is associative–commutative) and a structural finding (the acceptance gate forces apart the simultaneous swap and the time-separated bundle that `and` conflates). We argued that the discipline addresses, by construction, the execution-level failure modes that have dogged DeFi, that its native audit trail and pre-execution compliance proofs lighten the post-2008 regulatory burden and make the substrate a credible internal settlement engine for regulated institutions, and we sketched the spatial-behavioral type system that the calculus's own logic generator makes available as the next layer of robustness. A description language tells you what a contract is worth. A settlement language on this substrate tells you, before a single token moves, that it will settle.

Acknowledgements

This note develops joint work conducted in extended dialogue with Anthropic's Claude, which served as a sounding board throughout — re-expressing the combinator families in the cost-accounted calculus, surfacing the located-stack correction and the overloading of `and`, and pressing on the atomicity and resource-sufficiency arguments. The ideas and any errors are the author's.

References

[1] S. Peyton Jones, J.-M. Eber, J. Seward. Composing contracts: an adventure in financial engineering (functional pearl). In *ACM SIGPLAN International Conference on Functional Programming (ICFP)*, Montreal, 2000.

[2] S. Peyton Jones, J.-M. Eber. How to write a financial contract. In J. Gibbons and O. de Moor, eds., *The Fun of Programming*, Palgrave Macmillan, 2003.

[3] L. G. Meredith, M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science* 141(5):49–67, 2005.

[4] L. G. Meredith, M. Radestock. Namespace logic: a logic for a reflective higher-order calculus. In *Trustworthy Global Computing (TGC)*, LNCS 3705:353–369, Springer, 2005.

[5] L. G. Meredith. Cost-accounted rho calculus: a spectral decomposition of phlogiston. F1R3FLY.io, 2026.

[6] L. G. Meredith. Continued interactive GSLTs and the cost endofunctor: a construction one level up from cost-accounted Rholang. F1R3FLY.io, 2026.

[7] L. G. Meredith et al. Rholang specification. F1R3FLY.io / RChain Cooperative, 2017–2026.

[8] L. G. Meredith. MeTTaIL (Rholang 1.4): a language for defining languages. F1R3FLY.io, 2025–2026.

[9] R. Milner. *Communicating and Mobile Systems: the π-Calculus.* Cambridge University Press, 1999.

[10] M. Hennessy, R. Milner. Algebraic laws for nondeterminism and concurrency. *Journal of the ACM* 32(1):137–161, 1985.

[11] J.-Y. Girard. Linear logic. *Theoretical Computer Science* 50(1):1–101, 1987.

[12] J. C. Reynolds. Separation logic: a logic for shared mutable data structures. In *IEEE Symposium on Logic in Computer Science (LICS)*, 55–74, 2002.

[13] P. W. O'Hearn, J. C. Reynolds, H. Yang. Local reasoning about programs that alter data structures. In *Computer Science Logic (CSL)*, LNCS 2142:1–19, Springer, 2001.

[14] L. Caires, L. Cardelli. A spatial logic for concurrency (part I). *Information and Computation* 186(2):194–235, 2003.

[15] M. S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.

[16] M. Stay, L. G. Meredith. Representing operational semantics with enriched Lawvere theories. In *Higher-Dimensional Rewriting and Applications (HDRA)*, 2017. arXiv:1704.03080.

[17] N. G. de Bruijn. Lambda calculus notation with nameless dummies. *Indagationes Mathematicae* 34(5):381–392, 1972.

[18] N. Atzei, M. Bartoletti, T. Cimoli. A survey of attacks on Ethereum smart contracts (SoK). In *Principles of Security and Trust (POST)*, LNCS 10204, Springer, 2017.

[19] M. Bartoletti, R. Zunino. BitML: a calculus for Bitcoin smart contracts. In *ACM Conference on Computer and Communications Security (CCS)*, 2018.

[20] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, A. Juels. Flash Boys 2.0: frontrunning, transaction reordering, and consensus instability in decentralized exchanges. In *IEEE Symposium on Security and Privacy (S&P)*, 2020.

[21] P. Lamela Seijas, S. Thompson. Marlowe: financial contracts on blockchain. In *Leveraging Applications of Formal Methods, Verification and Validation (ISoLA)*, LNCS 11247:356–375, Springer, 2018.

[22] M. M. T. Chakravarty, J. Chapman, K. MacKenzie, O. Melkonian, M. Peyton Jones, P. Wadler. The Extended UTXO model. In *Financial Cryptography and Data Security Workshops (WTSC)*, LNCS 12063:525–539, Springer, 2020.

[23] F1R3FLY.io. F1R3Fi: development of DeFi using Peyton Jones–Eber–Seward contracts for financial engineering. Open-source software repository, F1R3FLY.io, 2026. https://github.com/F1R3FLY-io/F1R3Fi.

[24] United States Congress. Dodd–Frank Wall Street Reform and Consumer Protection Act. Pub. L. No. 111–203, 2010. (Title VII: over-the-counter derivatives reform.)

[25] European Parliament and Council. Regulation (EU) No 648/2012 on OTC derivatives, central counterparties and trade repositories (EMIR), 2012.

[26] European Parliament and Council. Directive 2014/65/EU (MiFID II) and Regulation (EU) No 600/2014 (MiFIR), 2014.

[27] Basel Committee on Banking Supervision. Basel III: a global regulatory framework for more resilient banks and banking systems. Bank for International Settlements, 2010 (rev. 2011).

[28] European Parliament and Council. Regulation (EU) 2023/1114 on markets in crypto-assets (MiCA), 2023.